

Ex. No. : 05

Date:

Register No.: 231701017

Name: HARISH BABU R

A Program To Implement Multi Layer Perceptron With Back Propagation

Aim:

To implement a **Multi Layer Perceptron with Backpropagation** to classify **Iris with low risk and high risk**

Requirements:

Jupyter Notebook with Python, Iris.csv dataset, and required Python libraries.

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

uploaded = files.upload()

df = pd.read_csv('Iris.csv')

X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']].values
X = (X - X.mean(axis=0)) / X.std(axis=0)

species_map = {
    'Iris-setosa': [1,0,0],
    'Iris-versicolor': [0,1,0],
    'Iris-virginica': [0,0,1]
}

Y = np.array([species_map[s] for s in df['Species']])

np.random.seed(0)

input_size = 4
hidden_size = 6
output_size = 3
```

```

W1 = np.random.randn(input_size, hidden_size)
b1 = np.zeros((1, hidden_size))

W2 = np.random.randn(hidden_size, output_size)
b2 = np.zeros((1, output_size))

lr = 0.01
epochs = 1000

def sigmoid(z):
    return 1/(1+np.exp(-z))

def sigmoid_derivative(a):
    return a*(1-a)

losses = []

for epoch in range(epochs):
    Z1 = np.dot(X, W1) + b1
    A1 = sigmoid(Z1)

    Z2 = np.dot(A1, W2) + b2
    A2 = sigmoid(Z2)

    loss = np.mean((Y - A2)**2)
    losses.append(loss)

    dA2 = (A2 - Y)
    dZ2 = dA2 * sigmoid_derivative(A2)

    dW2 = np.dot(A1.T, dZ2)
    db2 = np.sum(dZ2, axis=0, keepdims=True)

    dA1 = np.dot(dZ2, W2.T)
    dZ1 = dA1 * sigmoid_derivative(A1)

    dW1 = np.dot(X.T, dZ1)
    db1 = np.sum(dZ1, axis=0, keepdims=True)

    W2 -= lr * dW2
    b2 -= lr * db2

    W1 -= lr * dW1
    b1 -= lr * db1

    if epoch % 200 == 0:
        print(f"Epoch {epoch}, Loss: {loss}")

```

```

A1 = sigmoid(np.dot(X, W1) + b1)
A2 = sigmoid(np.dot(A1, W2) + b2)

predictions = np.argmax(A2, axis=1)
actual = np.argmax(Y, axis=1)

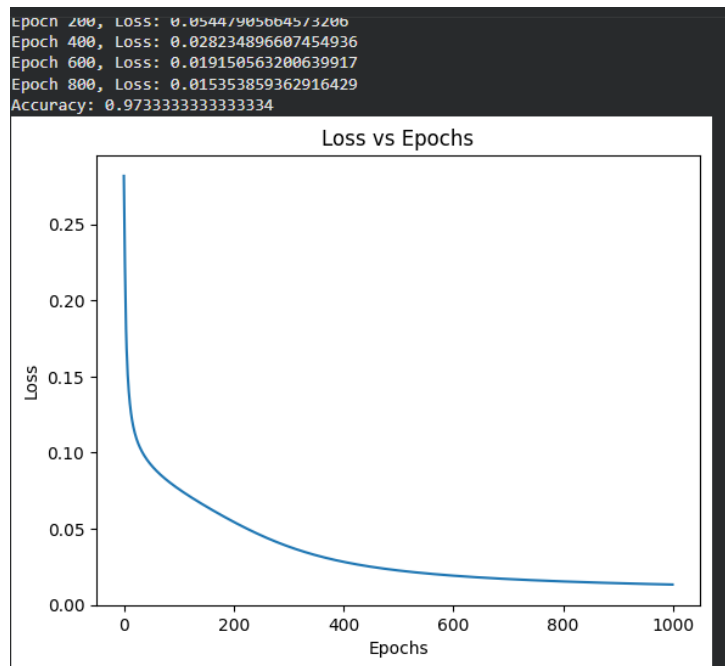
accuracy = np.mean(predictions == actual)
print("Accuracy:", accuracy)

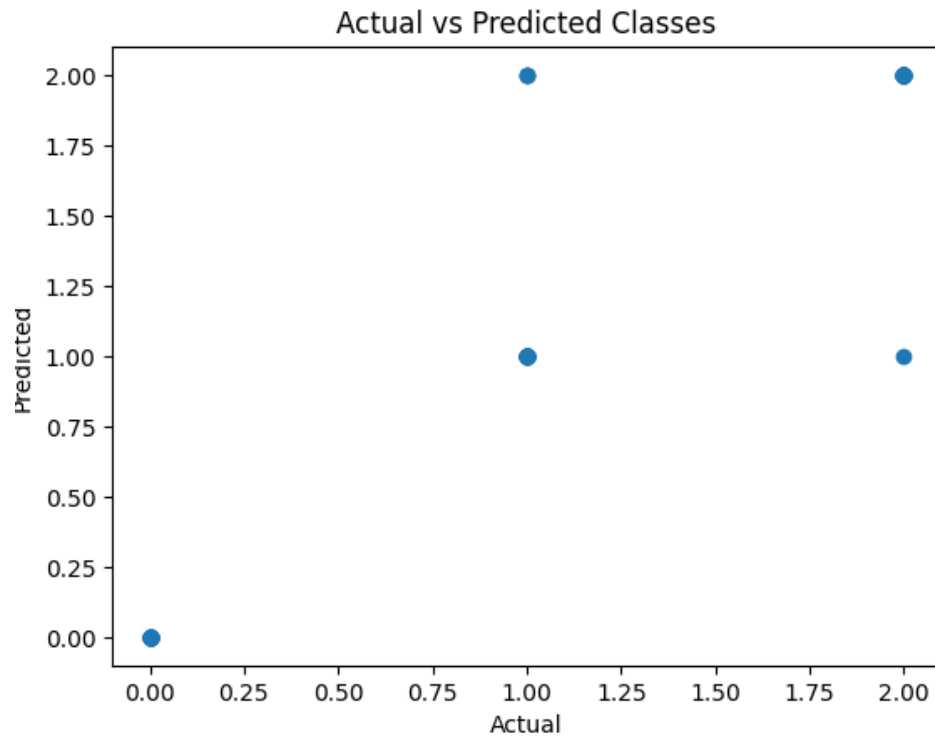
plt.figure()
plt.plot(losses)
plt.title("Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()

plt.figure()
plt.scatter(actual, predictions)
plt.title("Actual vs Predicted Classes")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()

```

OUTPUT:





Result:

Thus, the Multi Layer Perceptron with Backpropagation was successfully implemented and evaluated using the dataset.